COS 433: Cryptography

Reminders:

- Problem Set 1 due 9/24
- Problem Set 2 to be released on 9/24

Last time: CPA Security

IND-CPA Security

Challenger/Honest Party

Sampled once

k

$b \leftarrow \{0,1\}$

$m_0^{(\lambda)}, m_1^{(\lambda)}$

$\mathrm{Enc}(k, m_b; r)$

New $\mathbf{Enc}$ randomness each time

Adversary

b'

Win if $b' = b$

Definition: A secret-key encryption scheme with message length $n = n(\lambda)$ satisfies indistinguishability (IND-CPA) security if for all $t(\lambda) = \mathrm{poly}(\lambda)$,

For all polynomial-time  ,

$$\Pr[\mathrm{Win}] \leq \frac{1}{2} + \mathrm{negl}(\lambda)$$

IND-CPA Security, Alt. Definition

Challenger/Honest Party

k

$b \leftarrow \{0,1\}$

Adversary

b'

Win if $b' = b$

$m^{(\lambda)}$

$\mathrm{Enc}(k, m; r)$

$m_0^{(\lambda)}, m_1^{(\lambda)}$

Challenge: $\mathrm{Enc}(k, m_b; r)$

$m^{(\lambda)}$

$\mathrm{Enc}(k, m; r)$

For all polynomial-time  ,

$$\Pr[\mathrm{Win}] \leq \frac{1}{2} + \mathrm{negl}(\lambda)$$

IND-CPA Security

Claim: $\left|\Pr_{i+1}[\text{Win}] - \Pr_{i}[\text{Win}]\right| = \text{negl}(\lambda)$

Proof by contradiction: suppose  exists such that $\left|\Pr_{i+1}[\text{Win}] - \Pr_{i}[\text{Win}]\right| > \epsilon(\lambda)$

Game_i

k

$b \leftarrow \{0,1\}$

$m_0^{(\lambda)}, m_1^{(\lambda)}$

If $j \geq i+1$, $\text{Enc}(k, m_b; r)$

If $j < i+1$, $\text{Enc}(k, m_0; r)$

t times

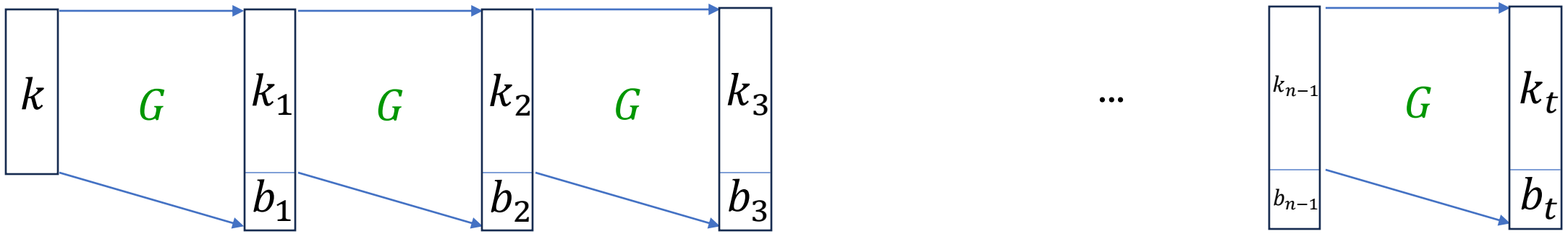
b'

Win if $b' = b$

violates the alt. definition!

Building CPA-secure Encryption

Partial Solution: Stream Cipher



Idea: To encrypt the i^{th} message, use b_i as a one-time pad (PRG length extension)

Secure! (won't prove)

But: parties must keep a counter for how many messages sent so far (and remember k_t).

"Stateful encryption" has many drawbacks (and does not satisfy our definition).

Building CPA-secure Encryption

k, m

$\mathrm{ct} = G(k) \oplus m$

k

$m' = G(k) \oplus \mathrm{ct}$

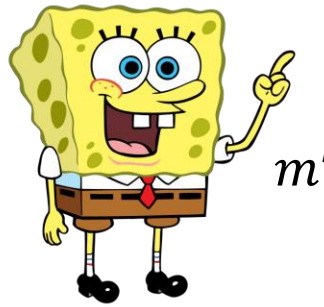
$G: \{0,1\}^{\lambda} \to \{0,1\}^{n}$

- We need to make use of randomness
- Problem: Bob doesn't get access to Alice's random coins… how to decrypt?
- Idea: can Alice make the coins public?

Building CPA-secure Encryption

k, m, r

$$\text{ct} = (r, F(k, r) \oplus m)$$



k

$$m' = F(k, \text{ct}_1) \oplus \text{ct}_2$$

$F: \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^n$

What property of F would guarantee CPA-security?

Pseudorandom Functions (PRFs)

Challenger

$b \leftarrow \{0,1\}$

$b = 0$: $k \leftarrow \{0,1\}^\lambda$

$b = 1$: $F \leftarrow \mathrm{Fun}(\{0,1\}^n \rightarrow \{0,1\}^m)$

$x \in \{0,1\}^n$

$b = 0$: $\mathrm{PRF}(k, x)$

$b = 1$: $F(x)$

Adversary

b'

Win if $b' = b$

Definition: An efficiently computable function family $\mathrm{PRF}: \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^m$ is a PRF family if:

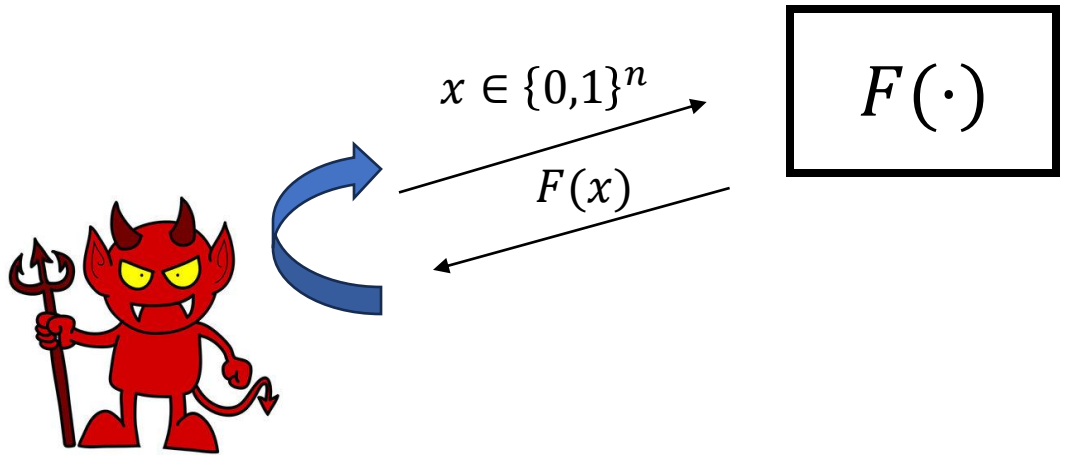
For all polynomial-time  ,

$$\Pr[\mathrm{Win}] \leq \frac{1}{2} + \mathrm{negl}(\lambda)$$

Pseudorandom Functions (PRFs)

Another view: "Oracle algorithms"

A poly-time oracle algorithm $A^{F(\cdot)}(1^\lambda)$ is an algorithm that can call a function $F(\cdot)$ on inputs of length $\mathbf{poly}(\lambda)$ at unit cost

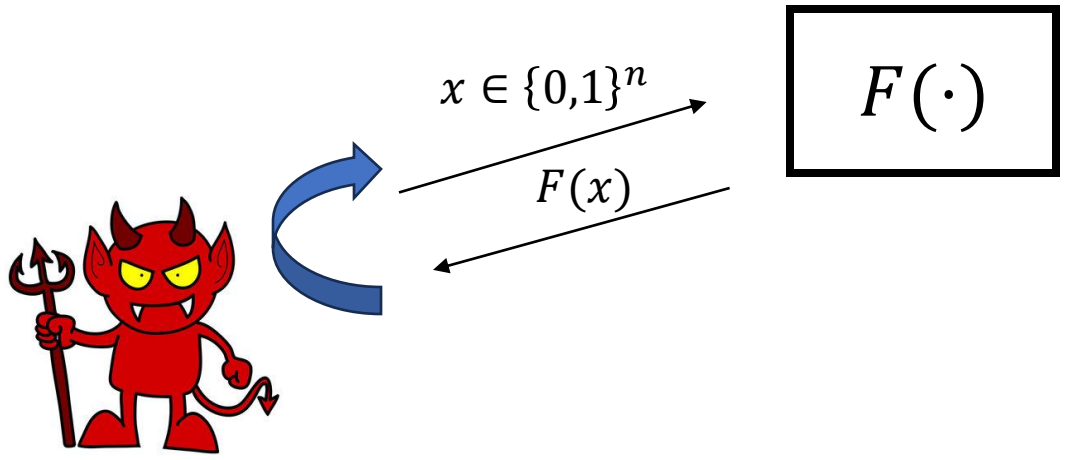


Definition: An efficiently computable function family $\mathbf{PRF}: \{0,1\}^\lambda \times \{0,1\}^n \to \{0,1\}^m$ is a PRF family if:

$$\text{For all polynomial-time } A^{F(\cdot)},\ A^{\mathbf{PRF}(k,\cdot)}(1^\lambda) \approx_c A^{F(\cdot)}(1^\lambda)$$

Pseudorandom Functions (PRFs)

Another view: "Oracle algorithms"

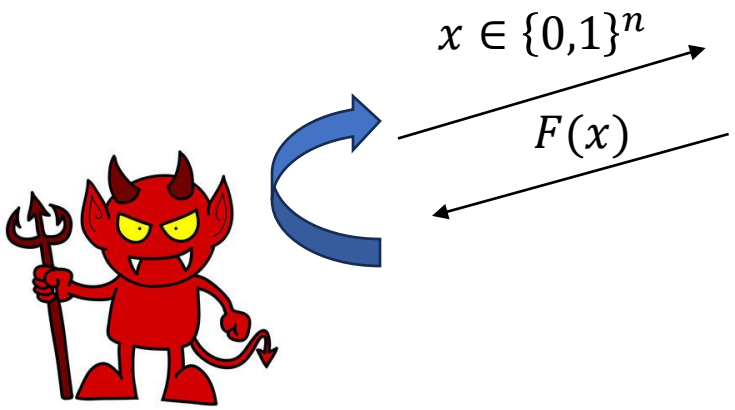


A poly-time oracle algorithm $A^{F(\cdot)}(1^\lambda)$ is an algorithm that can call a function $F(\cdot)$ on inputs of length $\mathbf{poly}(\lambda)$ at unit cost

Aside: can we efficiently compute $A^{F(\cdot)}(1^\lambda)$? A uniformly random $F(\cdot)$ is likely to be extremely hard to compute.

Observation: we can efficiently evaluate $A^{F(\cdot)}(1^\lambda)$ with a stateful algorithm

Pseudorandom Functions (PRFs)



$F(\cdot)$:

- Maintain a list of (x, y)
- If query is on the list, output y
- If not, sample y and add (x, y) to list

Aside: can we efficiently compute $A^{F(\cdot)}(1^\lambda)$? A uniformly random $F(\cdot)$ is likely to be extremely hard to compute.

Observation: we can efficiently evaluate $A^{F(\cdot)}(1^\lambda)$ with a stateful algorithm

Pseudorandom Functions (PRFs)

Challenger

$b \leftarrow \{0,1\}$

$b = 0$: $k \leftarrow \{0,1\}^\lambda$

$b = 1$: $F \leftarrow \mathrm{Fun}(\{0,1\}^n \rightarrow \{0,1\}^m)$

$x \in \{0,1\}^n$

$b = 0$: $\mathrm{PRF}(k, x)$

$b = 1$: $F(x)$

Adversary

b'

Win if $b' = b$

Definition: An efficiently computable function family $\mathrm{PRF}: \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^m$ is a PRF family if:

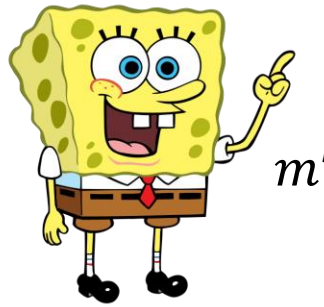
For all polynomial-time  ,

$$\Pr[\mathrm{Win}] \leq \frac{1}{2} + \mathrm{negl}(\lambda)$$

Building CPA-secure Encryption

k, m, r

$$\text{ct} = (r, \text{PRF}(k, r) \oplus m)$$



$m' = F(k, \text{ct}_1) \oplus \text{ct}_2$

k

$\text{PRF}: \{0,1\}^\lambda \times \{0,1\}^\lambda \to \{0,1\}^n$

Theorem: if **PRF** is a PRF family, then this encryption scheme is CPA-secure!

Proof: Hybrid argument & security reduction!

Building CPA-secure Encryption

k

$b \leftarrow \{0,1\}$

$m_0^{(\lambda)}, m_1^{(\lambda)}$

r , $\mathrm{PRF}(k, r) \oplus m_b$

t times

b'

Win if $b' = b$

$\approx_c$

by PRF security (reduction)

F

$b \leftarrow \{0,1\}$

$m_0^{(\lambda)}, m_1^{(\lambda)}$

r , $F(r) \oplus m_b$

t times

b'

Win if $b' = b$

Building CPA-secure Encryption

$m_0^{(\lambda)}, m_1^{(\lambda)}$

r , s

t times

b'

Win if $b' = b$

$\approx_{\text{negl}}$

$b \leftarrow \{0,1\}$

$m_0^{(\lambda)}, m_1^{(\lambda)}$

r , $F(r) \oplus m_b$

t times

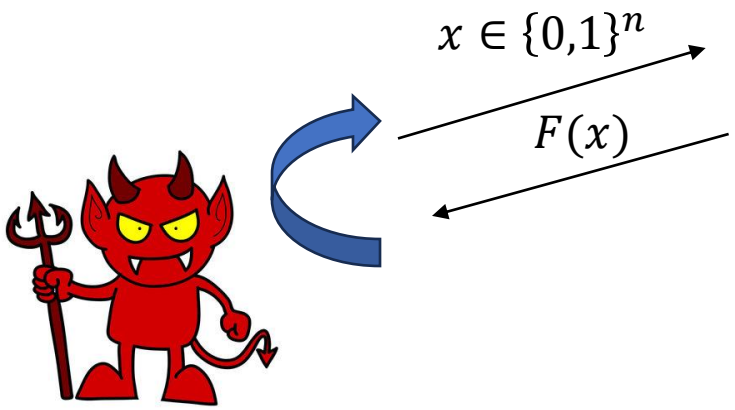
b'

Win if $b' = b$

F

$b \leftarrow \{0,1\}$

Pseudorandom Functions (PRFs)



$F(\cdot)$:

- Maintain a list of (x, y)
- If query is on the list, output y
- If not, sample y and add (x, y) to list

If we repeatedly call $F(r_1), \ldots F(r_t)$ for independent $r_1, \ldots, r_t$,
we get independent $y_1, \ldots, y_t$ unless some $r_i = r_j$.

Building CPA-secure Encryption

k

$b \leftarrow \{0,1\}$

$m_0^{(\lambda)}, m_1^{(\lambda)}$

r , s

(set $|r| = \lambda$ so that the t samples of r are distinct!)

t times

b'

Win if $b' = b$

$\approx_{\mathbf{negl}}$

F

$b \leftarrow \{0,1\}$

$m_0^{(\lambda)}, m_1^{(\lambda)}$

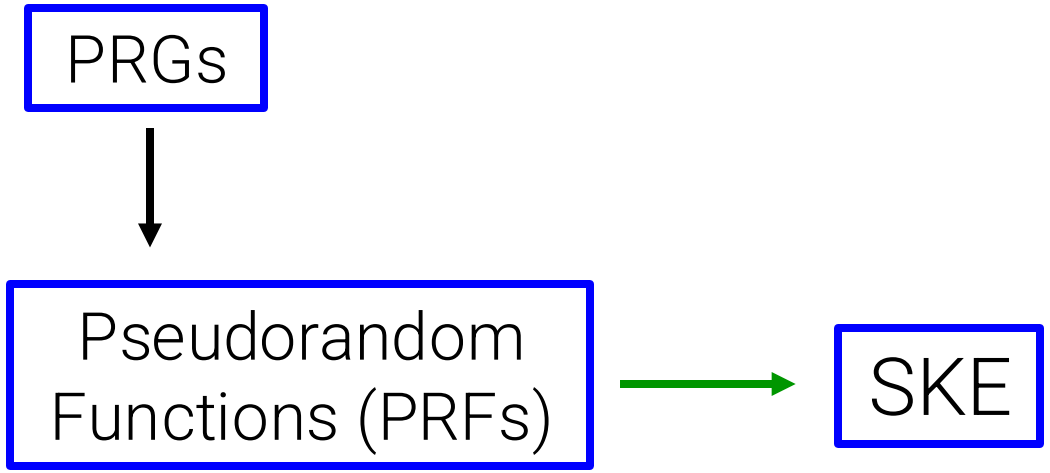
r , $F(r) \oplus m_b$

t times

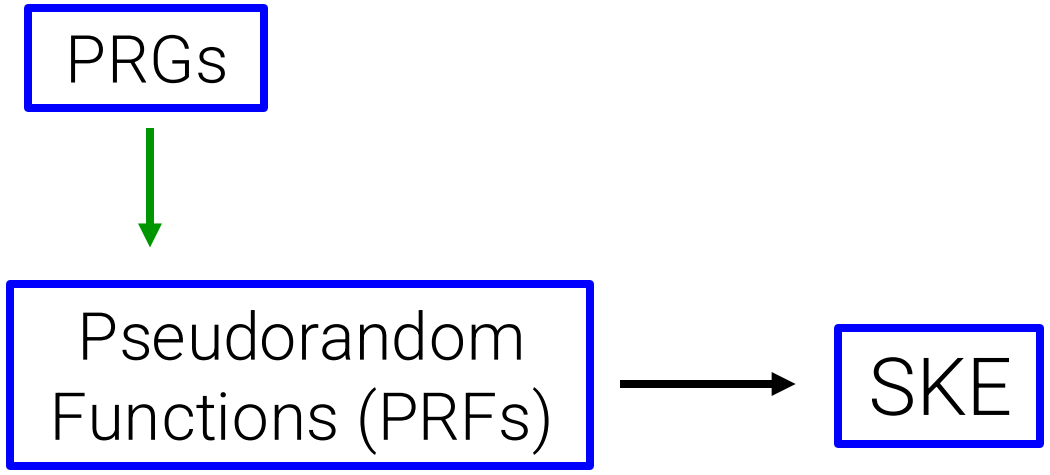
b'

Win if $b' = b$

Plan for this week



Plan for this week



Building PRFs

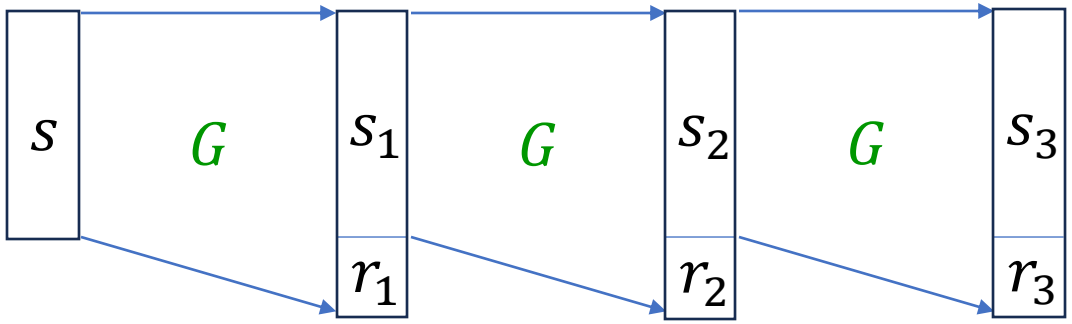
Theorem (GGM86): if $\mathbf{PRG}$s exist, then so do PRFs!

Goldreich – Goldwasser – Micali

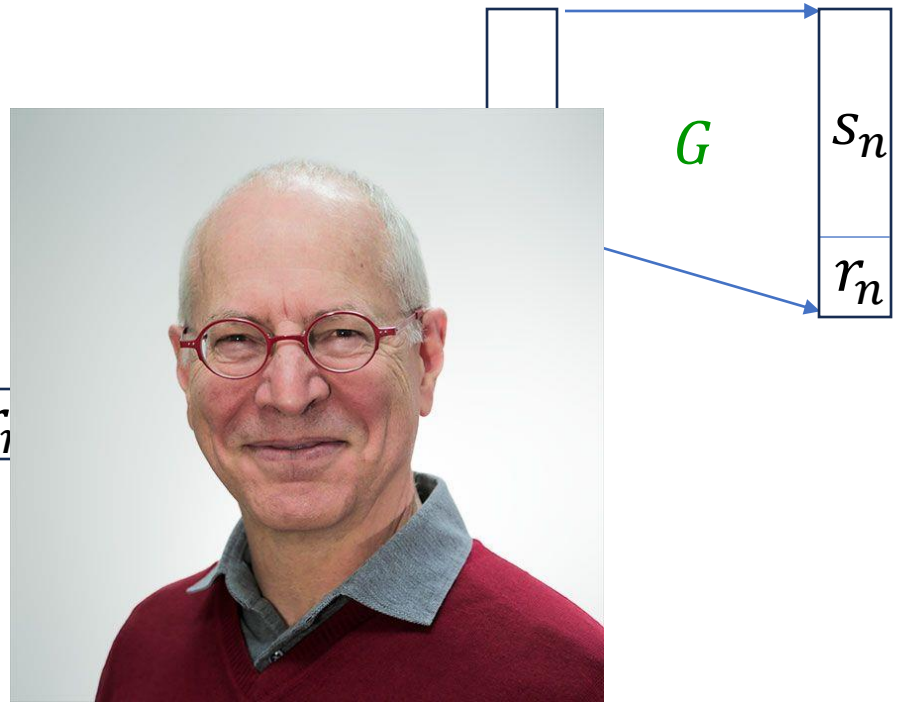
Building PRFs

Theorem (GGM86): if $\mathbf{PRG}$s exist, then so do PRFs!

Proof: Length extension??



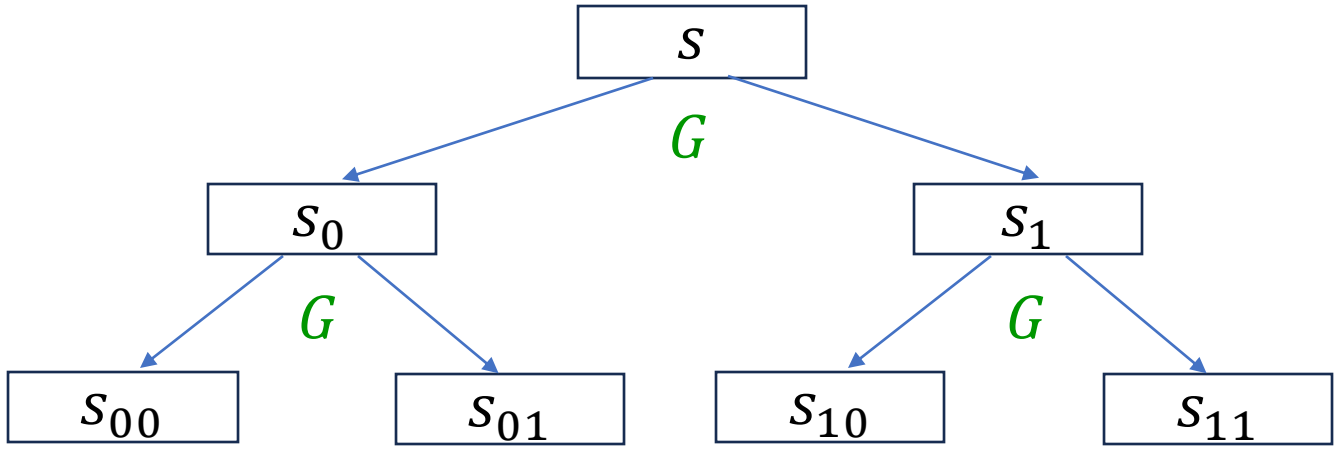
$r = r_1 \ldots r_n$



We are missing… a data structure!

Building PRFs

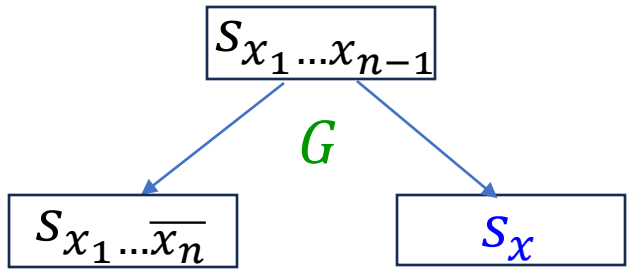
Let $G: \{0,1\}^{\lambda} \to \{0,1\}^{2\lambda}$ denote a length doubling PRG



Layer i: 2^i vertices

…

(suppose $x_n = 1$)



Define $\mathbf{PRF}_s(x) = s_x$

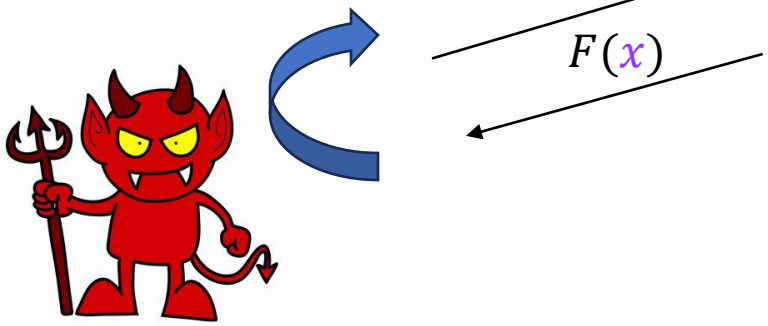
Building PRFs

Let $G: \{0,1\}^{\lambda} \rightarrow \{0,1\}^{2\lambda}$ denote a length doubling PRG

Defining $G(s) = \big(G_0(s), G_1(s)\big)$, this means:

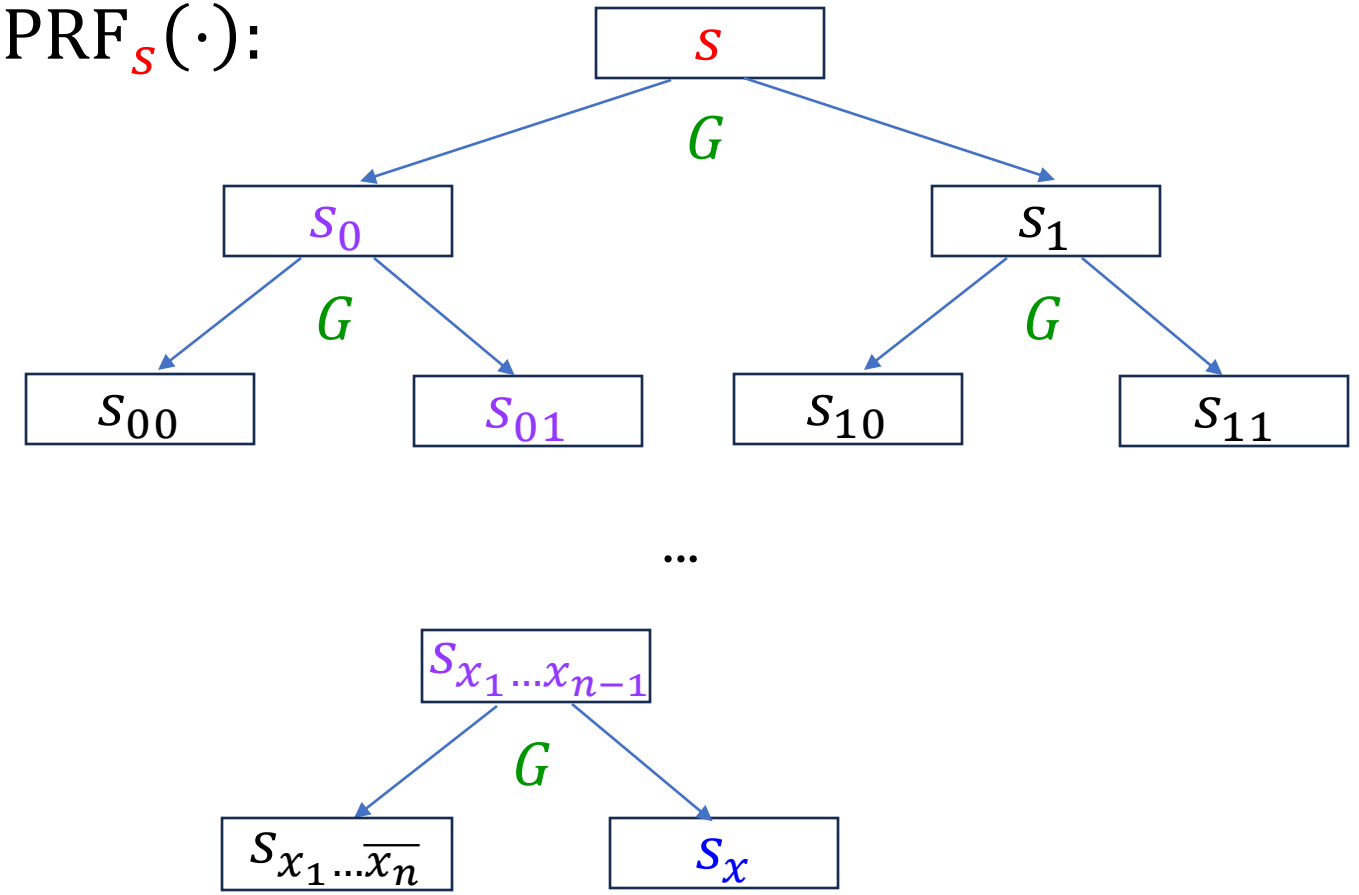
$$\mathrm{PRF}_s(x) = G_{x_n}\left(G_{x_{n-1}}\left(\ldots\left(G_{x_1}(s)\right)\right)\right)$$

Even though tree has exp size, any leaf value $\mathrm{PRF}_s(x)$ can be computed in polynomial time!



How do we prove pseudorandomness?

$\mathrm{PRF}_s(\cdot)$:



Continue next time